

Operational semantics for Prolog with Cut in Rocq and its application to determinacy analysis

Davide Fissore  

Université Côte d’Azur, Inria, France

Enrico Tassi  

Université Côte d’Azur, Inria, France

Abstract

We mechanize two operational semantics for PROLOG with cut and prove them equivalent, in the Rocq prover. The first semantics closely models the ELPI’s implementation, it is based on a stack of choice points and is well suited for studying optimizations employed by PROLOG abstract machines. The second semantics is instead based on and-or trees, in which the branches pruned by the cut operator are made explicit.

We use the tree-based semantics (1) to prove properties about the effect of the cut operator in the evaluation of choice points, since the rules from classical logic do not apply, and (2) to reason about the determinacy analysis introduced in Elpi version 3.0, which statically identifies computations that produce at most one result.

2012 ACM Subject Classification Theory of computation → Operational semantics

Keywords and phrases Semantics, Prolog, Elpi, Determinacy Analysis, Rocq, Coq

Digital Object Identifier 10.4230/LIPIcs.CVIT.2016.23

Funding *Davide Fissore*: This work has been supported by the French government, through the France 2030 investment plan managed by the Agence Nationale de la Recherche, as part of the “UCA DS4H” project, reference ANR-17-EURE-0004.

1 Introduction

ELPI is a dialect of λ PROLOG (see [16, 17, 7, 12]) used as an extension language for the ROCQ prover (formerly the COQ proof assistant). ELPI has become an important infrastructure component: several projects and libraries depend on it [13, 3, 4, 22, 8, 9]. Other remarkable libraries include the Hierarchy-Builder library-structuring tool [5] and Derive [20, 21, 11], a program-and-proof synthesis framework with industrial applications at SkyLabs AI.

Starting with version 3, ELPI is equipped with a static analysis for determinacy [10] to help users control backtracking. ROCQ users are familiar with functional programming, but not necessarily with logic programming; in particular, uncontrolled backtracking is a common source of inefficiency and makes debugging harder. The determinacy checker allows the user to assert which predicates should be considered deterministic and then checks that these predicates behave like functions, i.e., they commit to their first solution and leave no *choice points* (places where backtracking could resume). A choice point correspond to an suspended *alternative* in the execution trace. In the following we use *choice point* and *alternative* as synonyms.

This paper reports our first steps towards a mechanization, in the ROCQ prover, of the determinacy analysis from [10]. We focus on the control operator *cut*, which is useful to restrict backtracking but makes the semantics depart from a pure logical reading.

We formalize two operational semantics for PROLOG with cut. The first is a stack-based semantics that closely models ELPI’s implementation and is similar to the semantics mechanized by Pusch in ISABELLE/HOL [18] and to the model of Debray and Mishra [6, Sec. 4.3]. This stack-based semantics is a good starting point to study further optimizations



© Jane Open Access and Joan R. Public;
licensed under Creative Commons License CC-BY 4.0

42nd Conference on Very Important Topics (CVIT 2016).

Editors: John Q. Open and Joan R. Access; Article No. 23; pp. 23:1–23:17

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

```

func f int -> int. % f is a deterministic predicate. The two rules
f 1 2.             % have non-verlapping heads, i.e. 1 != 2
f 2 3.
pred r int -> int. % r is a non-deterministic predicate. The heads
r 0 0.             % overlap: the query for 0 has three applicable
r 0 1.             % rules.
r 0 2 :- !.
func g int -> int. % g is deterministic: if the first rule succeeds, the
g A C :- r A B, f B C, !. % second one is cut away, as well as the choice
g D F :- f D E, f E F.    % points left by the call to r.

```

■ **Figure 1** Running example of a ELPI program

used by standard PROLOG abstract machines [23, 1], but it makes reasoning about the scope of cut difficult. To address this limitation we introduce a tree-based semantics in which the branches pruned by cut are explicit and we prove the two semantics equivalent. Using the tree-based semantics, we then prove some properties about the impact of evaluating the cut in disjunctive queries. For example, unlike in classical logic, $q_1 \vee q_2$ is not equivalent to $q_2 \vee q_1$, since q_2 may be cut-away by q_1 .

Finally, we use the formal semantics to prove the correctness of a static determinacy analysis [10]. Intuitively, nondeterminism arises when a computation can be completed by applying two different rules. We exclude this situation in two ways. Firstly, at most one rule should be used on a given query. This condition is satisfied if (i) the rules have non-overlapping heads: if two heads do not overlap in the inputs they accept, then no query can unify with both, or if (ii) we account for the presence of cut in rule premises: once a rule whose premises contain a cut succeeds, all remaining rules are discarded. Secondly, each rule of a deterministic predicate should (i) either call functions, this ensure that no choice point is left after the execution of the premises of the chosen rule, (ii) or call relations provided that they are followed by the cut operator, so that any allocated choice points preceding the cut are discarded.

We show that if every rule defining a deterministic predicate passes this analysis, then any call to that predicate leaves no choice points.

2 Preliminaries: syntax and informal semantics of cut

Both semantics share the same notion of program, written $\mathbb{P}$. Figure 1 is our running example. A program consists of a list of rules $\mathbf{R}$ together with a signature environment $\mathbf{sigT}$. Besides the arity of each predicate, a signature specifies which arguments are inputs and which are outputs, and whether the predicate is deterministic. Inputs come before outputs.

A rule consists of a head term $\mathbf{Tm}$ and a list of premises. Each premise is an `Atom`, i.e. either the cut operator or a predicate call. Terms $\mathbf{Tm}$ include predicate symbols, data, and

```

Inductive Tm := ...
| Pred of P | Var of V.
Inductive sig := NonDet | Det.
Definition sigT :=
  P -> sig * nat * nat.

```

■ **Figure 2** Syntax of ELPI programs

```

Inductive A := cut | call : Tm -> A.
Record R := { head : Tm; premises : list A }.
Record P := { rules : list R; sig : sigT }.
Definition Σ := {fmap V -> Tm}.

```

■ **Figure 3** Syntax of ELPI programs

variables. In the concrete syntax, variables are written with capital letters, and predicate application follows the λ -calculus style (no parentheses).

The set of variables $\mathbf{V}$ is countable. We represent substitutions Σ as finitely supported maps from variables to terms, using the finmap library [15]. The empty substitution is written ε while the composition of two substitutions σ_1 and σ_2 with disjoint supports as $\sigma_1 + \sigma_2$.

The backchaining operation applies all rules to a query term; it is central to both semantics.

Definition $\text{bc} : \mathbb{P} \rightarrow \mathcal{F}_V \rightarrow \text{Tm} \rightarrow \Sigma \rightarrow \mathcal{F}_V * \text{list } (\Sigma * \text{list } \mathbf{A}) :=$

Since a rule may be applied multiple times, its variables must be freshened before each application. Accordingly, the backchain function takes a program, the set of variable names used so far ($\mathcal{F}_V$), a query term, and the current substitution. It returns an updated set of variable names together with the list of alternatives, i.e. the rules that apply. More precisely, for each applicable rule it returns the rule premises together with an updated substitution obtained by unifying the rule head with the query term.

In our mechanization we abstract the unifier and its properties, namely

```
Record Unif := {
  unify : Tm → Tm → Σ → option Σ;
  matching : Tm → Tm → Σ → option Σ;
}.
```

TODO: state axioms and explain matching; cite Dale's unification results. In this paper we assume the reader is familiar with first-order unification.

2.1 The cut operator

The cut operator in ELPI implements the *hard cut* (see, e.g., the documentation of SWI-PROLOG [19]). Whenever the backchaining function bc returns a list of alternatives, the first one is explored immediately, while the others are suspended as *choice points*. Within a given alternative, premises are processed from left to right. When a cut is encountered, it discards (i) all choice points created by bc for the current call and (ii) all choice points created while executing the premises preceding the cut.

For example, consider the program P in Figure 1 and the query $\mathbf{g} \ 0 \ \mathbf{R}$. Both rules for $\mathbf{g}$ apply to this query. Backchaining therefore returns the following two alternatives:

$[(\sigma_1, [\mathbf{r} \ \mathbf{A} \ \mathbf{B}, \ \mathbf{f} \ \mathbf{B} \ \mathbf{C}, \ !]), (\sigma_2, [\mathbf{f} \ \mathbf{D} \ \mathbf{E}, \ \mathbf{f} \ \mathbf{E} \ \mathbf{F}])]$

with $\sigma_1 := \{\mathbf{A} \mapsto 0, \mathbf{C} \mapsto \mathbf{R}\}$ and $\sigma_2 := \{\mathbf{D} \mapsto 0, \mathbf{F} \mapsto \mathbf{R}\}$. For simplicity, we choose an example where variable refreshing plays no role, so we do not discuss the set $\mathcal{F}_V$ any further.

Execution continues with the first premise of the first alternative, namely $\mathbf{r} \ \mathbf{A} \ \mathbf{B}$ under σ_1 , i.e. $\mathbf{r} \ 0 \ \mathbf{B}$. The remaining alternatives are stored as choice points. Backchaining $\mathbf{r} \ 0 \ \mathbf{B}$ returns $[(\sigma_3, []), (\sigma_4, []), (\sigma_5, [!])]$ where $\sigma_3 := \sigma_1 + \{\mathbf{B} \mapsto 0\}$; $\sigma_4 := \sigma_1 + \{\mathbf{B} \mapsto 1\}$ and $\sigma_5 := \sigma_1 + \{\mathbf{B} \mapsto 2\}$.

The next step of interpretation continues with $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under σ_3 , that is, $\mathbf{f} \ 0 \ \mathbf{R}$, and leaves $[(\sigma_4, []), (\sigma_5, [!])]$ as new choice points. This time, backchaining fails (no rule for $\mathbf{f}$ matches input 0). We therefore backtrack to the most recently introduced choice point and retry $\mathbf{f} \ \mathbf{B} \ \mathbf{C}$ under σ_4 , i.e. $\mathbf{f} \ 1 \ \mathbf{R}$. This succeeds with $\sigma_6 := \sigma_4 + \{\mathbf{R} \mapsto 2\}$.

We now reach the cut. At this point there are still two unexplored alternatives: one from the third rule for $\mathbf{r}$ and one from the second rule for $\mathbf{g}$ (respectively, σ_5 and σ_2). Both are discarded, because they were created when, or after, the first rule for $\mathbf{g}$ was selected.

```

Inductive tree :=
  | KO | OK                                     (* failure and success *)
  | Todo of Atom                               (* unexplored branch *)
  | Or of option tree &  $\Sigma$  & tree             (*  $A \vee B_\sigma$  *)
  | And of tree & list Atom & tree.            (*  $A \wedge_{B_0} B$  *)

Inductive tag := Expanded | CutBrothers | Failed | Success.
Definition step :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow (\mathcal{F}_v * \text{tag} * \text{tree}) :=$ 
Definition prune :  $\mathbb{B} \rightarrow \text{tree} \rightarrow \text{option tree} :=$ 
Inductive runT :  $\mathbb{P} \rightarrow \mathcal{F}_v \rightarrow \Sigma \rightarrow \text{tree} \rightarrow \text{option } (\Sigma * \text{option tree}) \rightarrow \mathbb{B} \rightarrow \mathcal{F}_v \rightarrow \text{Prop} :=$ 

```

■ **Figure 4** Signatures of the tree-based semantics

(In contrast, a cut does not discard choice points created earlier; in this example, there are none.)

The final solution is: $\{A \mapsto 0, B \mapsto 1, C \mapsto R, R \mapsto 2\}$.

In Sections 3 and 4, we provide fully mechanized operational semantics for PROLOG with cut. They differ in how they represent the ongoing computation, in particular how alternatives are stored and how they are pruned by cut.

Notational conventions In the following section we note $\mathbb{B}$ for the boolean type, and $\top$ and $\perp$ for **true** and **false**. The option instances are noted $\bullet t$ for *Some* t and $\square$ for *None*. Following the Mathematical Components library, on which we depend, we use $\mathbf{x}.1$ and $\mathbf{x}.2$ to denote the first and second projection applied to the pair $\mathbf{x}$.

3 And-Or Tree semantics

An And-Or tree is a stratified, variable-width tree. It consists of two kinds of layers that alternate: a disjunctive layer is followed by a conjunctive layer and vice versa. At the root (a query), the tree records a list of alternatives (an Or-layer); for each alternative, it records a list of subqueries to be solved (an And-layer). Intuitively, solving a query requires solving one alternative in the Or-layer, but all subqueries in the corresponding And-layer.

The tree is built incrementally. The **Todo** node represents an unexplored branch (an **Atom**). When the atom is a **call**, we use the backchaining procedure from Section 2 to compute the two layers to attach to the tree: alternatives form the Or-layer, and the premises of each applicable rule form the corresponding And-layer. This expansion step is performed by the **step** procedure described in Section 3.2.

The Rocq data type for And-Or trees is given in Figure 4. In addition to **Todo**, it provides two distinguished nodes, **KO** and **OK**, which represent respectively the smallest failed and successful tree.

The *Or* node, written $A \vee B_\sigma$, represents the addition of an alternative B_σ to an existing disjunction A . The left subtree A is optional and is absent once that branch has been fully explored. The right alternative is paired with a substitution σ , which becomes the current substitution when backtracking to B .

The *And* node, written $A \wedge_{B_0} B$, represents the addition of a subquery B to be solved after A . We call B_0 the *reset point* for B : it is used to restore the initial state of B when backtracking occurs within A . An example is shown in Section 3.3.

A graphical representation of a tree is shown in Figure 6b. For compactness, we depict *And* and *Or* nodes as n -ary (rather than binary), with children associating to the right. The

```

Fixpoint next  $\sigma$   $t$  :  $\Sigma$  * tree :=
  match  $t$  with
  | Todo _ | KO | OK  $\Rightarrow$  ( $\sigma$ ,  $t$ )
  | Or  $\square$   $\sigma'$  B  $\Rightarrow$  next  $\sigma'$  B
  | Or  $\bullet$ A _  $\Rightarrow$  next  $\sigma$  A
  | And A _ B  $\Rightarrow$ 
    let ( $\sigma'$ , pA) := next  $\sigma$  A in
    if pA == OK then next  $\sigma'$  B
    else ( $\sigma'$ , pA)
  end.

Definition next_subst  $\sigma$   $t$  := (next  $\sigma$   $t$ ).1.
Definition next_tree  $t$  := (next  $\varepsilon$   $t$ ).2.

Definition success  $t$  := next_tree  $t$  == OK.
Definition failed  $t$  := next_tree  $t$  == KO.
Definition incomplete  $t$  :=
  if next_tree  $t$  is Todo _ then  $\top$  else  $\perp$ .

```

■ Figure 5 *next* and derived functions

145 *KO* node acts as the terminating element of an *Or* list, while *OK* is the terminating element
 146 of an *And* list.

147 3.1 The tree-based semantics

148 The tree is constructed incrementally by two recursive functions, *step* and *prune*. Both
 149 traverse the tree depth-first, left-to-right. The node to be explored in a tree is retrieved
 150 thanks to the *next_tree* (in Figure 5). In the snippet we also identify special kinds of trees:
 151 depending on the *next_tree*, we distinguish between *success*, *failed* and *incomplete* trees.
 152 Since all functions must terminate in Rocq, we define their transitive closure as the inductive
 153 relation *runT*.

154 Intuitively, *runT* iterates calls to *step* on incomplete trees to evaluate the *Todo* node.
 155 Upon failed trees, it calls *prune* to find the next alternative and continues from there, if one
 156 exists. We detail *step*, *prune*, and *runT* in Sections 3.2–3.4.

157 In Figure 6 we mark with a black arrow the result of *next_tree*.

158 3.2 The *step* procedure

159 The *step* procedure takes as input a program, a set of variables, a substitution, and a tree,
 160 and returns an updated set of variables, a *tag*, and an updated tree.

161 The set of variables is assumed to contain all variables occurring in the tree. It is passed
 162 to backchain so that rules can be refreshed correctly.

163 The *tag* (see Figure 4) indicates which kind of step was performed. **Failure** and **Success**
 164 are returned for trees that satisfy, respectively, the *failed* and *success* tests. **CutBrothers**
 165 indicates the interpretation of a superficial cut: *step* was called on an *And*-layer and its
 166 *next_tree* is the cut atom. Finally, **Expanded** accounts for the interpretation of predicate calls
 167 and non-superficial cuts.

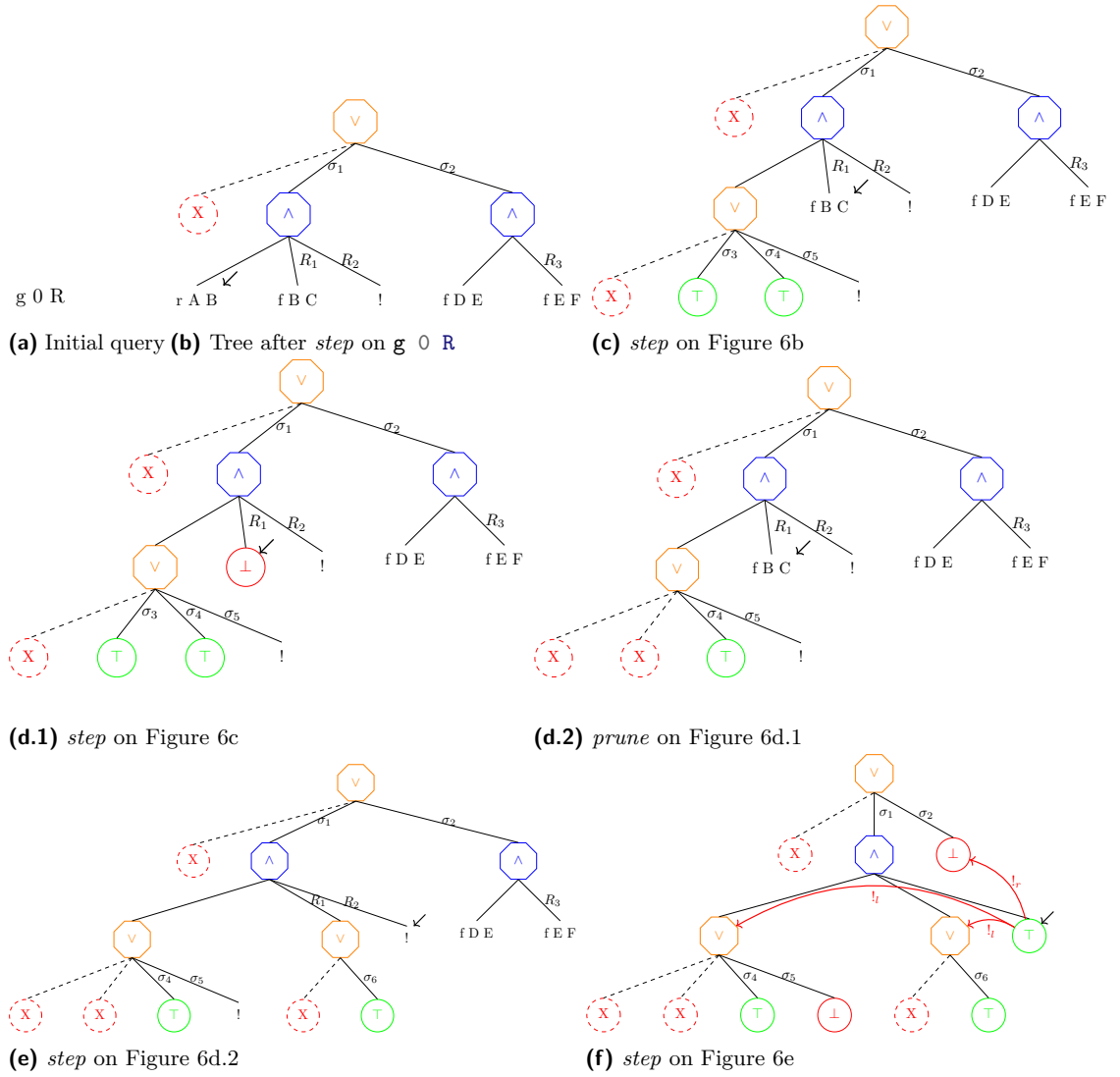
168 The *step* procedure evaluates atoms. In particular, it leaves the tree unchanged when the
 169 tree is *success* or *failed*.

170 **Lemma** succ_step_iff: $\forall p v \sigma t$, success $t \leftrightarrow$ step $p v \sigma t = (v, \text{Success}, t)$. (1)

171 **Lemma** fail_step_iff: $\forall p v \sigma t$, failed $t \leftrightarrow$ step $p v \sigma t = (v, \text{Failed}, t)$. (2)

172 *step* expands *Todo* nodes by calling backchain, which returns a list *l* of alternatives. If *l* is
 173 empty, then the current call is replaced by *KO*. Otherwise, it is replaced by a right-skewed

¹ The substitutions $\sigma_1 \dots \sigma_6$ are from Section 2.1. R_1, R_2 , and R_3 are reset points and correspond respectively to the lists of atoms [f Y Z, !], [!], and [f Y Z].



■ **Figure 6** Some tree representations¹

tree of *Or* nodes, where each leaf corresponds to a list of atoms $e \in l$. If e is empty, the leaf is *OK*; otherwise, it is a right-skewed tree of *And* nodes.

Figure 6b depicts the tree corresponding to the running example (Section 2.1) as it undergoes calls to *step*. We run query $q \ 0 \ R$ against the program P in Figure 1. Let c_0 , c_1 , and c_2 be the children of the root. By construction, c_0 is the $\square$ tree, while c_1 and c_2 correspond respectively to the premises of rules r_1 and r_2 in P . We need the initial $\square$ subtree so that we can attach a substitution to c_1 . In particular, c_1 (resp. c_2) is associated with substitution σ_1 (resp. σ_2), whose values are the same as in Section 2.1. Reset points are marked with the R symbol: $R_1 := [f \ Y \ Z, !]$, $R_2 := [!]$, and $R_3 := f \ Y \ Z$. Intuitively, they record the lists of atoms used to generate the corresponding subtree. Other examples of *step* triggering backchaining appear in Figures 6c–6e.

3.2.1 The interpretation of a cut

The step corresponding to a cut performs two actions: (i) the cut node is replaced by *OK*; and (ii) the subtrees in the scope of *Cut* are pruned. More precisely, (ii) prunes the left siblings of the *Cut* node (in the same *And*-layer, denoted $!_l$) and prunes the right uncles of *Cut* (in the one-level-up *Or*-layer, denoted $!_r$).

An example of the impact of cut is shown in Figure 6f. The red arrows indicate the scope of the cut and are labeled with $!_r$ and $!_l$ to indicate which pruning operation is applied to each pointed subtree. We note that the evaluation of a cut keeps the path leading to the cut node unchanged, in the sense that substitution σ_4 is preserved, while the path under substitution σ_5 (which also succeeds) is replaced by *KO*. This amounts to discarding the third rule of predicate r (see Section 2.1).

3.3 The prune procedure

When backchain returns an empty list, *step* produces a *failed* tree and the computation must backtrack. The *prune* procedure is responsible for producing the new tree from which the computation can continue.

In Figure 6d.1 we encounter a failure because no rule applies to the atom $f \ B \ C$ under substitution σ_3 , which maps B to 0. The *prune* procedure prunes the path leading to this failure and resets the current sub-query to its original shape. More precisely, it kills the *OK* node under σ_3 (since that branch has been fully explored and produced no solution), and then replaces the *KO* node with the information stored in the reset point R_1 . This restores the call $f \ B \ C$, which can then be reconsidered under substitution σ_4 .

Furthermore, *prune* serves another important purpose. Its behavior depends on the boolean flag it receives as input: *prune* $\perp A$ recursively removes all failures (if any) from A , whereas *prune* $\top A$ removes the first success in A . For example, the tree in Figure 6f contains a successful path that maps R to 2. Calling *prune* on this tree returns $\square$, since there are no alternatives in the tree after the success.

Some interesting properties of *prune* are shown below; they illustrate how *prune* complements *step* with respect to failed, success, and *path_atom* trees.

Lemma incomplete_prune_id: $\forall b \ t, \text{incomplete } t \rightarrow \text{prune } b \ t = \bullet t.$ (3)

Lemma prune_failedF: $\forall b \ t \ t', \text{prune } b \ t = \bullet t' \rightarrow \text{failed } t' = \perp.$ (4)

Lemma pruneFN_fail: $\forall t, \text{prune } \perp t = \square \rightarrow \text{failed } t.$ (5)

sottolineare le
sostituzioni

216 3.4 The *runT* inductive

217 The inductive relation *runT* relates four inputs to three outputs. The inputs are a program,
218 a set of variables, an initial substitution σ , and a tree t . The outputs are (i) an optional
219 result r of the form $\bullet(\sigma', t')$, (ii) a boolean b , and (iii) an updated set of variable names.

220 The main output is r : if $r = \square$, interpretation fails; otherwise σ' is the computed solution
221 substitution and t' represents the remaining, not-yet-explored alternatives. The boolean b
222 records whether a superficial cut was evaluated during execution, and the updated variable
223 set tracks freshly generated names.

224 *runT* has four cases, depicted as inference rules in Figure 7. (STOPT) If *next_tree* t is a
225 success, then the substitution σ' is extracted from t (starting from σ) and the input tree is
226 cleaned of its successful path. (STEPT) If *next_tree* t is an atom, then *step* is invoked to
227 evaluate t , and *runT* is called recursively on the updated tree. (BACKT) If *next_tree* t is a
228 failure, then *prune* is called to clear the failed path; if the resulting cleaned tree exists, *runT*
229 is called recursively on it. Finally, (FAILT) accounts for trees with no solutions.

230 These rules are deterministic:

231 **Lemma** *runT_det*: $\forall p \ v_0 \ \sigma \ t \ r \ r' \ b \ b' \ v \ v',$
 $\text{runT } p \ v_0 \ \sigma \ t \ r \ b \ v \rightarrow \text{runT } p \ v_0 \ \sigma \ t \ r' \ b' \ v' \rightarrow r' = r \wedge v' = v \wedge b' = b. \quad (6)$

232 The proof is by induction on the first *runT* derivation and is immediate, because the
233 rules are selected based on *next_tree* t . In particular, the hypotheses *success* t , *path_atom* t ,
234 and *failed* t are mutually exclusive. Moreover, by Lemma 6, the hypothesis of FAILT implies
235 *failed* t ; hence FAILT is exclusive with STOPT and STEPT, and it is also exclusive with
236 BACKT since they impose different requirements on the output of *prune*.

$$\begin{array}{c}
 \frac{\text{success } t \quad \text{next_subst } \sigma \ t = \sigma' \quad \text{prune } \top \ t = t'}{\text{runT } p \ v \ \sigma \ t \ \bullet(\sigma', t') \ \perp \ v} \text{STOPT} \\
 \\
 \frac{\text{incomplete } t \quad b' = (\text{tg} == \text{CutBrothers}) \parallel b \quad \text{step } p \ v \ \sigma \ t = (v', \text{tg}, t') \quad \text{runT } p \ v' \ \sigma \ t' \ r \ b \ v''}{\text{runT } p \ v \ \sigma \ t \ r \ b' \ v''} \text{STEPT} \\
 \\
 \frac{\text{failed } t \quad \text{prune } \perp \ t = \bullet t' \quad \text{runT } p \ v \ \sigma \ t' \ r \ n \ v'}{\text{runT } p \ v \ \sigma \ t \ r \ n \ v'} \text{BACKT} \\
 \\
 \frac{\text{prune } \perp \ t = \square}{\text{runT } p \ v \ \sigma \ t \ \square \ \perp \ v} \text{FAILT}
 \end{array}$$

■ **Figure 7** Rule system for *runT*

237 3.5 Properties of *runT*

238 Working with *runT* allows to prove some interesting properties about the behavior of the
239 cut operator wrt disjunction, i.e. the *Or* node. We have proven several properties that are
240 available at [this link](#). Below we show two of these lemmas.

241 **Lemma** *runSST_or*: $\forall p \ v \ v' \ \sigma \ \sigma' \ l \ l' \ \sigma_1 \ r,$
 $\text{runT } p \ v \ \sigma \ l \ \bullet(\sigma', \bullet l') \ \top \ v' \rightarrow$
 $\text{runT } p \ v \ \sigma \ (\text{Or } \bullet l \ \sigma_1 \ r) \ \bullet(\sigma', \bullet(\bullet l' \vee \text{KO}_{\sigma_1})) \ \perp \ v'. \quad (7)$

$$\begin{aligned}
& \text{Lemma runNT_or: } \forall p \ v \ v' \ \sigma \ t \ \sigma_1 \ t', \\
& \text{runT } p \ v \ \sigma \ t \ \square \top \ v' \rightarrow \\
& \text{runT } p \ v \ \sigma \ (\bullet t \vee t'_{\sigma_1}) \ \square \perp \ v'.
\end{aligned} \tag{8}$$

$$\begin{aligned}
& \text{Lemma run_orSST } p \ v \ v' \ \sigma \ \sigma' \ \sigma_1 \ l \ l' \ r \ r': \\
& \text{runT } p \ v \ \sigma \ (\bullet l \vee r_{\sigma_1}) \bullet(\sigma', \bullet(\bullet l' \vee r'_{\sigma_1})) \perp \ v' \rightarrow \\
& \exists b, \text{runT } p \ v \ \sigma \ l \bullet(\sigma', \bullet l') \ b \ v' \wedge r' = \text{if } b \text{ then } KO \text{ else } r.
\end{aligned} \tag{9}$$

In Lemma 7, we evaluate the tree l . This evaluation produces a success with signature σ' and a new tree l' . During the evaluation, a superficial cut in l is traversed. This implies that the evaluation of $\bullet l \vee_{\sigma_1} r$ returns σ' as substitution, and the alternatives are $\bullet l' \vee_{\sigma_1} KO$. The right-hand side of the *Or* node is replaced with *KO* by $!_r$.

In Lemma 8, we are in a similar scenario: t evaluates a superficial cut but, this time, produces a failure. This implies that the evaluation of $\bullet t \vee_{\sigma_1} t'$ also fails.

Finally, in Lemma 9, we evaluate $\bullet l \vee_{\sigma_1} r$. The evaluation succeeds, producing the result $\bullet(s_1, \bullet l' \vee_{\sigma_1} r')$. This means that l still has l' as unexplored alternatives. We deduce that the evaluation of l produces the substitution σ_1 with alternatives l' . However, we have no information about whether a superficial cut has been crossed in l . If this is the case, we know that r' is cut away by $!_r$; otherwise, r' has not been explored and is equal to r .

From these lemmas, it is clear how the order in which alternatives are evaluated changes the behavior of the program.

4 Stack semantics

We now introduce the stack semantics. The interpreter is close to that of the ELPI language and provides an operational semantics that is similar to the one proposed by Pusch in [18], which is in turn closely related to the semantics given by Debray and Mishra in [6, Section 4.3]. Pusch mechanized this semantics in Isabelle/HOL, together with several optimizations that are also present in the Warren Abstract Machine [23, 1].

We represent a state of the ELPI language thanks to the two mutual inductives shown below.

```

Inductive alts := nilA | consA of (Σ * goals) & alts
with goals := nilG | consG of (A * alts) & goals .

```

A stack is an enhanced two-dimensional list, called *alts* in the inductive. The outermost list represents a list of alternatives in disjunction, each accompanied by the substitution that should be used for their interpretation. The innermost list is a list of atoms, that is, the list of goals in conjunction. These goals are decorated with a pointer to an alternative list, which is used to keep track of where to jump when a cut is interpreted. We call these special alternatives the *cut-to* alternatives.

The stack interpreter is called *runE* and has the following type.

$$\text{Inductive runS } (p: \mathbb{P}) : \mathcal{F} \rightarrow \text{alts} \rightarrow \text{option } (\Sigma * \text{alts}) \rightarrow \text{Prop} := \tag{1}$$

The inductive *runE* represents a routine taking as input a set of variables and a list of alternatives and returning an option. $\square$ stands for interpretation with no solution, whereas $\bullet(\sigma, a)$ represent a successful interpretation with σ as solution and a as the list of non-yet-explored choice points.

The rule system for *runE* is given in Figure 8 and is made by 4 cases. We distinguish two main cases: if the list of alternatives is empty, (FAILE) we have a failure: we stop the computation and return $\square$. If the list of alternatives is the list $(\sigma_0, h) :: a$, the interpreter evaluates the first choice point h under the substitution σ_0 . If h is empty, (STOPE), then we

se riesco success
(A or B) = suc-
cess (B or A) if
no sup cut in A
and B

Dire che non ab-
biamo le due
info ritornate da
runT

Definition `save_gs` $a \ g \ b :=$
 $[\text{seq } (x, a) \mid x <- b] ++ g.$

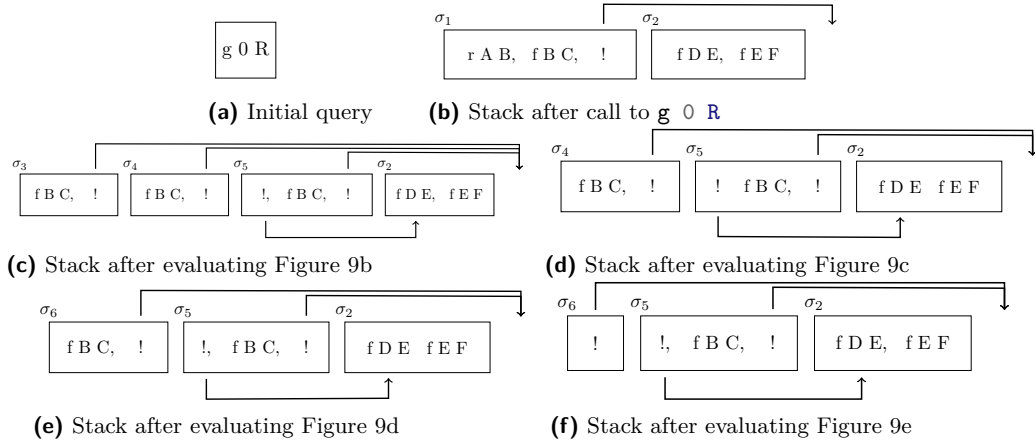
Definition `save_alts` $a \ g \ b :=$
 $[\text{seq } (x.1, \text{save_gs } a \ g \ x.2) \mid x <- b].$

Definition `stepE` $p \ v \ t \ \sigma \ a \ g :=$
 $\text{let } (v', r) := \text{bc } p \ v \ t \ \sigma \ \text{in}$
 $(v', \text{save_as } a \ g \ r).$

$$\frac{}{\text{runS } p \ v \ ((\sigma, \emptyset) :: a) \bullet (\sigma, a) \text{ STOPS}} \quad \frac{\text{runS } p \ v \ ((\sigma, g) :: ca) \ r}{\text{runS } p \ v \ ((\sigma, (\text{cut}, ca) :: g) :: _) \ r} \text{ CUTS}$$

$$\frac{\text{stepE } p \ v \ t \ \sigma \ a \ g = (v_1, a') \quad \text{runS } p \ v_1 \ (a' ++ a) \ r}{\text{runS } p \ v \ ((\sigma, (\text{call } t, _) :: g) :: a) \ r} \text{ CALLS} \quad \frac{}{\text{runS } p \ _ \ \emptyset \ \square \text{ FAILS}}$$

■ **Figure 8** Rule system for *runE*



■ **Figure 9** Example of *runE* execution

281 have a success, i.e. we return the input σ_0 and leave a as the future choice points. Otherwise,
 282 h is the list $(t, ca) :: g$ where t is the first atom to evaluate, ca is its cut-alternative list and g
 283 is the list of future conjuncts. If t is a cut (CUTE), we need to keep evaluating g under σ_0
 284 but we change the alternatives to ca . Finally, in the case of a call (CALLE), we backchain
 285 the call in the program. If rs is the list of rule premises returned by `bc`, then each list of
 286 premises if mapped to a list of goals (`goals`) have a as cut-alternative and the global list is
 translated to a list of alternative (`alts`).

Dire una parola
 su backtracking
 = tl if bc = nil

290 *Example of execution* We propose an example of the execution of *runE* in Figure 9. The
 291 arrows in the images represent the cut-alternative pointers of the cut. We have not drawn
 292 arrows coming out of atom-calls since the interpreter ignores the cut-alternatives in this case
 293 (cf. CALLE). The evolution of the stack in Figure 9 mirrors the example in Section 2.1: we
 294 evaluate the first goal of the first alternative. During backchaining, we add a pointer to
 295 the remaining alternatives to each new cut operator. If a cut is evaluated, we replace the
 296 remaining alternatives with the cut-to alternatives. For example, in Figure 9f, evaluating
 the cut discards the second and third elements in the stack. Backtracking is performed by
 simply removing the first alternative from the stack, as shown in Figure 9d.

297 *Remark* We want finally to point out how the lemma that we were able to prove in
 298 Section 3.5 are not easy to be expressed in the stack semantics. This is because there is
 299 no sufficient structure in the stack to understand what is the scope of the cut operator.
 300 For example, in a stack made of these alternatives $a_0, a_1 \dots a_n$, we have no clear idea of

```

Fixpoint valid_tree A :=
  match A with
  | Todo _ | OK | KO => T
  | Or □ _ B => valid_tree B
  | Or •A _ B => valid_tree A &&
    ((B == KO) || B.base_or B)
  | And A B0 B => valid_tree A &&
    if success A then valid_tree B
    else B == big_and B0
  end.

Fixpoint t2l t σ (cp: alts) : alts :=
  match t with
  | OK      => [(σ, ∅)]
  | KO      => ∅
  | TA a    => [(σ, [(a, ∅)])]
  ...

```

(a) Valid tree

(b) Tree to list

■ **Figure 10** Valid tree and Tree to list

what happens if the first alternative a_0 succeeds: there could be a cut in a_0 but, is this cut superficial? What is its impact wrt the alternatives $a_1 \dots a_n$? More then that, it is also possible that the state is hill-formed and the cut does not point to a suffix in the list of alternatives.

5 Equivalence of the two semantics

In order to relate the two semantics we have to relate the computations states, i.e., the And-Or tree and the stack of alternatives. In particular we define a translation from trees to stacks, called *t2l*. We do not explicitly provide the converse translation, an economy allowed by the proof technique we employ, inspired by [2].

Before describing the translation of trees to stacks (in Section 5.2) we need to define the notion of *valid tree*. The **tree** datatype indeed contains trees that cannot be generated by **runT** via **step** or **prune**, for example all the trees that do not correspond to a depth-first left-to-right tree construction strategy.

5.1 Valid trees (*valid_state*)

The class of valid trees is characterized by the function shown in Figure 10a.

While all base cases of tree are considered valid, we need to analyze carefully the cases for the *Or* and *And* constructors.

For the *Or* constructor, we distinguish two cases depending on whether the left hand side is present. If it is not present, then the right hand side must be a valid tree. Otherwise, the the left hand side must be a valid tree and the right hand side must either be the *KO* tree or be unexplored. The first case occurs when the right hand side is cut out by the evaluation of a superficial cut present in the left hand side. In the latter case we use the **base_or** predicate to check that **B** is the right-skewed tree formed by a disjunction of conjunctions that is generated after a successful backchain to a **call**.

For the *And* constructor, the left hand side is required to be a valid tree. The shape of the right hand side depends on whether the left hand side is a success. If it is not successful, then the right hand side must not be explored, i.e. **B** must be the right-skewed tree containing conjunctions of premises atoms. This is enforced using the **big_and** produce that generates a tree out of the reset point B_0 , the same procude used to transform each alternative output by **bc** into the And-layer of the resulting tree. When the left hand side is successful, then the right hand side may have been explored, and consequently must be a valid tree.

t	6a	6b	6c	6d.1 and 6d.2	6e	6f
$t2l\ t \in \emptyset$	9a	9b	9c	9d	9e	9f

■ **Table 1** Tree to list from Figure 6 and Figure 9

5.2 From trees to stacks ($t2l$)

DAVVERO?

The $t2l$ recursive function translates a tree to a stack. For space constraints Figure 10b only gives the base cases, while we describe the recursive cases in the following text.

TODO

The function $t2l$ takes the tree t_0 , a substitution σ_0 and a list of choice points cp . These choice points represent alternatives that appears at the same level of the current tree, such as, for example, the right-siblings of a *Or* node. The substitution TODO .

TODO

The *OK* node represent one succesful alternative, thereofre it is translated into a singleton list with the empty list of goals. The *KO* node represent a failure, i.e. it is mapped to the empty list of alternatives. Finally, atoms are mapped to one alternative containing a single goal with the empty cut-to alternative list. They cannot point to cp since they are not right-uncle subtrees, i.e. *EXPLAIN*.

superficial?

The translation of $A \vee B_\sigma$ creates two list of alternatives, one for the A and the other for B . The alternatives of B are valid choice points for A and should be passed to the translation of A . The two list of alternatives can be concatenated and, each atom, should add cp as new cut-to alternatives: A and B are one level lower wrt cp and therefore the cut they have should point to cp .

dirlo meglio
all'inizio

The translation of $A \wedge_{B_0} B$ starts with the translation of A . If the translation of A is an empty list we return the empty list of alternatives without even touching B nor B_0 , since an empty translation for A indicates failure and this in turn implies the failure of the entire conjunction. When the translation of A is a list $(\sigma_0, g) :: a$ the B node represent the continuation of the first alternative g , whereas B_0 is the continuation of each alternative in a .

5.2.1 Example of $t2l$

In Table 1 we point out the result of calling $t2l$ on the trees in Figure 6. The first raw contains the image reference of then trees and the second raw contains the reference of the images from the stacks in Figure 9. For the 3rd column in the table, we see that $t2l$ is not injective. There are different trees that maps to the same stack. In fact, there are transitions in $runT$ that are not visible in $runE$.

2) prendere
un albero, un
nodo interno,
dire come t2l e
chiamata in quel
caso e metterè
accanto lo stack
corrispondente

5.3 Equivalence proof

The tree-based semantics exposes more information so to be usable to express the lemmas in Section 3.5. For the equivalence this information is irrelevant, hence define a version of $runT$ that hides it.

Definition $runT' \ p \ v \ \sigma \ t \ r := \exists \ b \ v', \ runT \ p \ v \ \sigma \ t \ r \ b \ v'.$ (2)

From Table 1, we observe that, upon backtracking, $runT$ may perform more reduction steps than $runE$ before returning a solution or a failure. These transitions are called silent transitions and must be skipped when proving the equivalence between the two semantics. Therefore, we state the following lemma.

► **Lemma 1** ($pruneF_t2l$).

$$\begin{aligned} \forall \ t \ t' \ b, \ valid_tree \ t \rightarrow failed \ t \rightarrow prune \ b \ t = \bullet t' \rightarrow \\ \forall \ l \ \sigma, \ t2l \ t \ \sigma \ l = t2l \ t' \ \sigma \ l. \end{aligned}$$

369 **Proof.** By induction on the tree t .

◀ dire di v

370 The first direction of the equivalence states that the execution of a valid tree is matched
371 by the execution of the stack obtain by translating the tree and moreover the unexplored
372 alternatives returned as a tree match the ones returned as a stack.

373 ▶ **Theorem 2** (tree_to_elpi).

$$\begin{aligned} \forall p \ t \ \sigma \ r, \text{ let } v &:= \text{vars_tree } t \text{ `/' `vars_sigma } \sigma \text{ in} \\ \text{valid_tree } t &\rightarrow \\ \text{runT'} \ p \ v \ \sigma \ t \ r &\rightarrow \\ \text{let } a &:= \text{t2l } t \ \sigma \ \emptyset \text{ in} \\ \text{let } r' &:= \text{if } r \text{ is } \bullet(\sigma', t) \text{ then } \bullet(\sigma', \text{t2l } (\text{odflt } KO \ t) \ \sigma \ \emptyset) \\ &\quad \text{else } \square \text{ in} \\ \text{runS } p \ v \ a \ r'. \end{aligned}$$

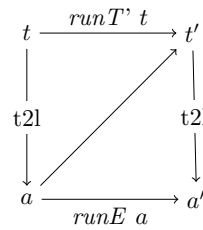
374 **Proof.** We proceed by induction on the execution of runT' . There are four cases to consider.
375 The first case arises from **STOPT**; it deals with successful trees. A successful tree must start
376 with an empty list, and therefore we conclude using **STOPE**. The case for **STEPT** requires
377 treating two subcases: *step* evaluates either a cut or a call. Accordingly, we apply **CUTE** or
378 **CALLE**, followed by the induction hypothesis. The case for **BACKT** is silent: it represents
379 the non-injective behavior of t2l ; however, thanks to Lemma 1 and the induction hypothesis,
380 the conclusion is proved. The last case arises from the derivation **FAILT**. It is easily proved
381 using **FAILE** and the fact that if *prune* returns $\square$ when trying to erase failure in t , then t2l
382 applied to t returns the empty list. ◀

383 The converse direction is stated following [2]: instead of using a function to translate a
384 stack into a tree it states that any tree that translates to the given stack has an equivalent
385 execution, i.e. gives the same solution and returns a unexplored tree that translates to the
386 unexplored stack.

387 ▶ **Theorem 3** (elpi_to_tree).

$$\begin{aligned} \forall p \ v \ a \ r, \text{ runS } p \ v \ a \ r &\rightarrow \\ \forall \sigma \ t, \text{ valid_tree } t &\rightarrow \text{t2l } t \ \sigma \ \emptyset = a \rightarrow \\ \text{if } r \text{ is } \bullet(\sigma', a') &\text{ then} \\ \exists t', \text{ runT'} \ p \ v \ \sigma \ t \bullet(\sigma', t') &\wedge \text{t2l } (\text{odflt } KO \ t') \ \sigma \ \emptyset = a' \\ \text{else } \text{runT'} \ p \ v \ \sigma \ t &\square. \end{aligned}$$

388 **Proof.** The proof is based on the idea explained in [2, Section 3.5.1] and
depicted in the figure on the right: we have a state t whose translation is
 a , we proceed by induction on the execution of runE . This gives us not
only the output a' but that hypothesis that it exists a tree t' such that
its translation to the stack state is a' . This proof strategy is indirectly
providing a kind of l2t function by combining the execution of the runE
and t2l . ◀



389 Stating the converse direction in a symmetric way would have requires a function l2t
390 transforming an stack a to a tree t . Writing this function is far from trivial, since the
391 structure of the tree is lost by *save_alts* and *run* that intuitively keep the state into a big
392 disjunction of conjunctions by distributing conjunctions over disjunctions. Moreover one
393 would need to define a notion of valid stack that is equally intricate.

6 Case study: determinacy analysis

An interesting application of the tree semantics is the determinacy checking that was added in version 3 of the ELPI language [10]. The checker is meant to receive the signature of the predicates declared by the user and then verify that the rules of a deterministic predicate will return at most one solution, we define this particular kind of predicate as functions. Thanks to this static verification, the user can better control unexpected backtracking at compile time: if an incoherence is rilevata then a fatal error explain why the current program may break determinacy. A function behaves deterministically if two main conditions are respected: *mutual exclusion* guarantees that at most one rule can be successfully used on a given query, whereas *rule checking* ensure that each rule does not creates choice points, e.g. by calling non-deterministic predicates.

The syntax of the signatures in Figure 1 indicates that **f** and **g** are expected to be functions, as indicated by the **func** keyword. The **->** symbol separates inputs from outputs; therefore, all these predicates are expected to take an **int** as input and return an **int** as output.

The signatures of the program are stored in its **sig** projection of the program P passed to the interpreter, and are encoded as described in [10], tahs is, similar to the code in Figure 1, we have arrows decorated with an input mode for term application, base types (to encode, for example, *int*), and two special symbols to distinguish deterministic from non-deterministic predicates.

A program execution behaves deterministically if given a program, a substitution and a tree to the *runT* inductive, then either the execution fails, or, if it succeeds, then the tree of alternatives is $\square$. Below we define **is_det**.

Definition **is_det** $p \sigma v t := \forall r, \text{runT}' p v \sigma t r \rightarrow r = \square \vee \exists \sigma, r = \bullet(\sigma, \square)$.

The theorem we want to prove is the following.

► **Lemma 4** (**det_check_call**).

$$\forall p \sigma t, \text{let } v := \text{vars_tm } t \text{ '/' } \text{vars_sigma } \sigma \text{ in} \\ \text{check_P } p \rightarrow \text{tm_is_det } p.(sig) t \rightarrow \text{is_det } p \sigma v (\text{Todo } (call t)).$$

With **check_P** being the function checking the program wrt mutual exclusion and rule checking and **tm_is_det** being the function testing if the term head refers to a rigid constant with deterministically type. The proof of this lemma should be done on the derivation of the program execution. However, in this definition, the induction hypothesis is too weak, since in we would have to prove ... but we will not be able to show that the ... is deterministic.

To solve the problem, we need to work on “deterministic trees” (**det_tree**), show that if a tree respect this condition, and then prove the following theorem which is more generic then **det_check_call**.

► **Lemma 5** (**det_check_tree**).

$$\forall \sigma v p t, \text{vars_tree } t \text{ '<=' } v \rightarrow \text{vars_sigma } \sigma \text{ '<=' } v \rightarrow \\ \text{check_P } p \rightarrow \text{det_tree } p.(sig) t \rightarrow \text{is_det } p \sigma v t.$$

Since **det_check_call** is an instance of **det_check_tree**, its proof is now trivial.

As an important remark, proving the same lemma with the stack semantics, it is difficult to correctly define the predicate **det_stack**, checking for the deterministic behavior of a generic stack, since, similar to Section 3.5, the lack of structure make this goal hard. We need therefore an intermediate data structure, like the tree to encompass this problem.

use coq syntax
instead of elpi?

7 Related work

We studied a PROLOG semantics in which input arguments are *matched*, rather than unified, against rule heads. This choice agrees with the ELPI interpreter, which is in turn inspired by B-Prolog’s “matching-clauses” [24]. The two semantics we presented (and proved equivalent) also apply to standard PROLOG if one forces all predicate signatures to have only output arguments.

For determinacy analysis, matching input arguments affects the development only as follows: axiom XXX does not require the query q to be ground. Adapting our results to standard PROLOG therefore requires two changes. First, to add the corresponding groundness hypothesis to XXX. Second, to discharge this premise when XXX is used. Typically this requires an additional static analysis that checks whether the program is well-moded. Well-moded predicates produce ground outputs assuming ground inputs; hence, starting from a ground initial query, well-moded programs behave exactly as if input arguments were matched, and hence the proofs in Section 6 still apply.

The only mechanization of Prolog’s semantics we could find is the one by Pusch in ISABELLE/HOL [18] that is based on the model of Debray and Mishra [?, Sec. 4.3]. Their work start from that semantics and refines it by introducing some optimizations usually found in the Warren abstract machine [23, 1]. They do not use the semantics to prove properties on the execution of programs as we do in our use case, hence they do not face the difficulty described in Section 4 that pushed us to develop an more explicit semantics (with respect to cut). In some way our work is complementary, showing the stack based semantics equivalent to a more optimized one.

Another relevant work are the ones by King [14], but we could not find their Rocq source code. Their semantics is denotational and cannot easily cover all the features our paper [10] does, but would have been sufficient for this first step.

The proof technique we use to relate the two semantics is inspired by [2] and we thank Y.Bertot for insightful discussions on the topic. In [2] Bertot relates bla bla, witout never constructing a decompiler, exactly as we never build a list to tree function.

FINIRE

8 Conclusion

We do this. This is a first step for XXXX. We need to add substitutions. The missing part of the proof becomes related to abstract interpretation, since the det types for a lattice and one has to prove that the concrete runtime values variables take agree with the abstraction computed statically. This proof is ongoing and somehow orthogonal to the current one since the cut operator and the HO feature do not interfere with eachother in complex ways.

References

- 1 Hassan Ait-Kaci. *Warren’s Abstract Machine: A Tutorial Reconstruction*. The MIT Press, 08 1991. doi:10.7551/mitpress/7160.001.0001.
- 2 Yves Bertot. A certified compiler for an imperative language. Technical Report RR-3488, INRIA, September 1998. URL: <https://inria.hal.science/inria-00073199v1>.
- 3 Valentin Blot, Denis Cousineau, Enzo Crance, Louise Dubois de Prisque, Chantal Keller, Assia Mahboubi, and Pierre Vial. Compositional pre-processing for automated reasoning in dependent type theory. In Robbert Krebbers, Dmitriy Traytel, Brigitte Pientka, and Steve Zdancewic, editors, *Proceedings of the 12th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2023, Boston, MA, USA, January 16-17, 2023*, pages 63–77. ACM, 2023. doi:10.1145/3573105.3575676.

- 479 **4** Cyril Cohen, Enzo Crance, and Assia Mahboubi. Trocq: Proof transfer for free, with or
480 without univalence. In Stephanie Weirich, editor, *Programming Languages and Systems*, pages
481 239–268, Cham, 2024. Springer Nature Switzerland.
- 482 **5** Cyril Cohen, Kazuhiko Sakaguchi, and Enrico Tassi. Hierarchy Builder: Algebraic hierarchies
483 Made Easy in Coq with Elpi. In *Proceedings of FSCD*, volume 167 of *LIPIcs*, pages 34:1–34:21,
484 2020. URL: [https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2020.34)
485 34, doi:10.4230/LIPIcs.FSCD.2020.34.
- 486 **6** Saumya K. Debray and Prateek Mishra. Denotational and operational semantics for prolog. *J.*
487 *Log. Program.*, 5(1):61–91, March 1988. doi:Debray1988.
- 488 **7** Cvetan Dunchev, Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. ELPI: fast,
489 embeddable, λ Prolog interpreter. In *Proceedings of LPAR*, volume 9450 of *LNCS*, pages
490 460–468. Springer, 2015. URL: <https://inria.hal.science/hal-01176856v1>, doi:10.1007/
491 978-3-662-48899-7_32.
- 492 **8** Davide Fissore and Enrico Tassi. A new Type-Class solver for Coq in Elpi. In *The Coq*
493 *Workshop*, July 2023. URL: <https://inria.hal.science/hal-04467855>.
- 494 **9** Davide Fissore and Enrico Tassi. Higher-order unification for free!: Reusing the meta-
495 language unification for the object language. In *Proceedings of PPDP*, pages 1–13. ACM, 2024.
496 doi:10.1145/3678232.3678233.
- 497 **10** Davide Fissore and Enrico Tassi. Determinacy checking for elpi: an higher-order logic
498 programming language with cut. In Nada Amin and Joaquín Arias, editors, *Practical Aspects*
499 *of Declarative Languages*, pages 77–95, Cham, 2026. Springer Nature Switzerland.
- 500 **11** Benjamin Grégoire, Jean-Christophe Léchenet, and Enrico Tassi. Practical and sound equality
501 tests, automatically. In *Proceedings of CPP*, page 167–181. Association for Computing
502 Machinery, 2023. doi:10.1145/3573105.3575683.
- 503 **12** Ferruccio Guidi, Claudio Sacerdoti Coen, and Enrico Tassi. Implementing type theory
504 in higher order constraint logic programming. In *Mathematical Structures in Computer*
505 *Science*, volume 29, pages 1125–1150. Cambridge University Press, 2019. doi:10.1017/
506 S0960129518000427.
- 507 **13** Robbert Krebbers, Luko van der Maas, and Enrico Tassi. Inductive Predicates via Least
508 Fixpoints in Higher-Order Separation Logic. In Yannick Forster and Chantal Keller, editors,
509 *16th International Conference on Interactive Theorem Proving (ITP 2025)*, volume 352 of *Leib-*
510 *niz International Proceedings in Informatics (LIPIcs)*, pages 27:1–27:21, Dagstuhl, Germany,
511 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. URL: [https://drops.dagstuhl.](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27)
512 [de/entities/document/10.4230/LIPIcs.ITP.2025.27](https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2025.27), doi:10.4230/LIPIcs.ITP.2025.27.
- 513 **14** Jael Kriener and Andy King. Redalert: Determinacy inference for prolog. volume 11, pages
514 537–553. Cambridge University Press, 2011.
- 515 **15** math-comp. finmap — finite maps for mathcomp, 2026. URL: [https://github.com/](https://github.com/math-comp/finmap)
516 [math-comp/finmap](https://github.com/math-comp/finmap).
- 517 **16** Dale Miller. A logic programming language with lambda-abstraction, function variables, and
518 simple unification. In *Extensions of Logic Programming*, pages 253–281. Springer, 1991.
- 519 **17** Dale Miller and Gopalan Nadathur. *Programming with Higher-Order Logic*. Cambridge
520 University Press, 2012.
- 521 **18** Cornelia Pusch. Verification of compiler correctness for the wam. In Gerhard Goos, Juris
522 Hartmanis, Jan van Leeuwen, Joakim von Wright, Jim Grundy, and John Harrison, editors,
523 *Theorem Proving in Higher Order Logics*, pages 347–361, Berlin, Heidelberg, 1996. Springer
524 Berlin Heidelberg.
- 525 **19** SWI-Prolog Documentation. Predicate `!/0` — cut (built-in predicate), 2026. Accessed via SWI-
526 Prolog PLDoc reference. URL: https://www.swi-prolog.org/pldoc/doc_for?object=!/0.
- 527 **20** Enrico Tassi. Elpi: an extension language for Coq (Metaprogramming Coq in the Elpi λ Prolog
528 dialect). In *The Fourth International Workshop on Coq for Programming Languages*, January
529 2018. URL: <https://inria.hal.science/hal-01637063>.

- 530 21 Enrico Tassi. Deriving proved equality tests in Coq-Elpi. In *Proceedings of ITP*, volume 141 of
531 *LIPICs*, pages 29:1–29:18, September 2019. URL: <https://inria.hal.science/hal-01897468>,
532 doi:10.4230/LIPICs.CVIT.2016.23.
- 533 22 Luko van der Maas. Extending the Iris Proof Mode with inductive predicates using Elpi.
534 Master’s thesis, Radboud University Nijmegen, 2024. doi:10.5281/zenodo.12568604.
- 535 23 David H.D. Warren. An Abstract Prolog Instruction Set. Technical Report Technical Note 309,
536 SRI International, Artificial Intelligence Center, Computer Science and Technology Division,
537 Menlo Park, CA, USA, October 1983. URL: [https://www.sri.com/wp-content/uploads/](https://www.sri.com/wp-content/uploads/2021/12/641.pdf)
538 [2021/12/641.pdf](https://www.sri.com/wp-content/uploads/2021/12/641.pdf).
- 539 24 Neng-fa Zhou. The language features and architecture of b-prolog. *Theory Pract. Log. Program.*,
540 12(1–2):189–218, January 2012. doi:10.1017/S1471068411000445.